AuroraRL

Abstract:

Nowadays, LLM Reinforcement Learning (RL) requires frequent synchronization of parameters between the trainer and distributed rollout actors. Since most organizations cannot afford tightly-coupled clusters, the alternative is loosely-coupled clusters.

Introduction:

So now we to solve problem that comes with the loosely-coupled clusters: Capacity, Resilience, and Accessibility.

- Capacity: Post-training job requirements can exceed what any single region can supply.
- Resilience: Decoupled deployments suffer from outages, and provider-side disruptions.
- Accessibility: cross-region and cross-provider links typically offer only 1-10Gbps bandwidth and exhibit high WAN latency and loss rate.

Background:

- ### HPC Cloud Premium

  SOTA RL systems assume an RDMA-connected HPC cluster within a single datacenter as the default model. However single-datacenter HPC model introduces both and access barriers. RDMA clusters typically charge a network premium. For example $39/hour. Also, RL may need more than one datacenter can offer. Beyond, access remains constrained. Academic institutions, government labs, and small organizations often cannot reserve the large GPU blocks.

- ### Challenges:

  - **Challenge 1 (C1): The commodity network barrier.**

    Decentralized deployments connect the Trainer and Actors over commodity networks (1-10Gbps cross-region, cross-cloud, or heterogeneous-instance links), where full model weight transfer dominates iteration time and throttles throughput regardless of GPU speed. For example, if we have 8B model (Qwen3-8B) in BF16, pure model size will be 16GB. The transfer will take 1.3 in an HPC fabric, within a 45 s generation window; 128 s on a 1Gbps link. It is beyond what even one-step asynchronous RL can hide.

  - **Challenge 2 (C2): Hardware and network heterogeneity.**

GPU instances range from older L40 and A100 GPUs to newer H100 hardware, with generation throughput differing by 2 to 3 times.
Network conditions are equally heterogeneous: Actors from nearby providers may achieve 5 to 10 Gbps bandwidth to the Trainer, while those across continents operate at 1 to 3 Gbps.

- **Challenge 3 (C3): Dynamic membership and failures.**

  Actors can disappear involuntarily through spot preemption, hardware or network failures, and regional outages; and join or leave deliberately as operators add capacity, shift regions, or resize the pool mid-run.

- **Why do existing systems fall short?**

  - **Colocated** systems assume datacenter fabrics, homogeneous hardware, and stable membership.
  - **Disaggregated** systems tolerate some heterogeneity but require ~80Gbps dedicated links and a stable cluster.
  - **Decentralized** systems reach internet scale and absorb churn but still broadcast full weights, hiding that cost via multi-step staleness that compromises model quality.

Design Principles:

Challenges get easier to solve if we dissect the RL workflow.

- **Empirical basis: fine-grained sparsity**

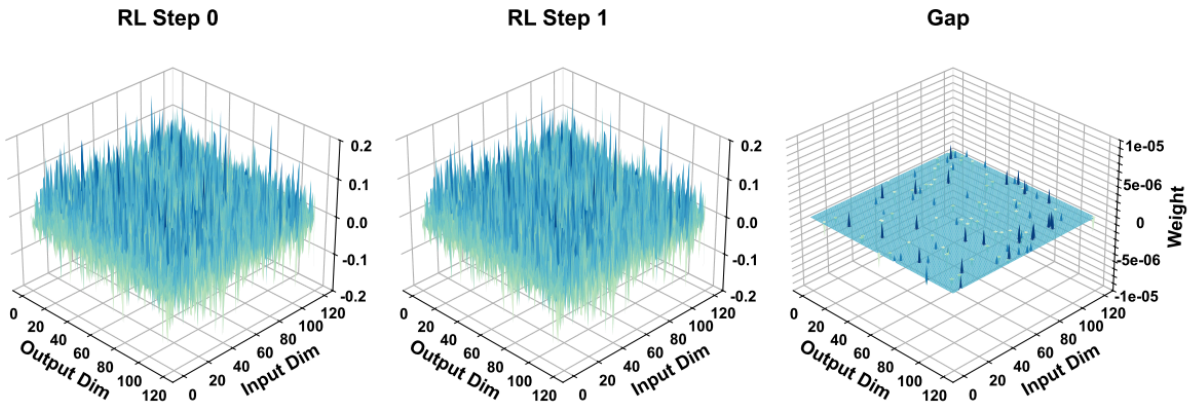
  An LLM consists of K parameter tensors $\{W^{(k)}\}_{k=1}^{K}$.
  Let $W_t^{(k)}$ and $W_{t+1}^{(k)}$ denote the values of tensor k before and after one RL step, and define the element-wise difference $\Delta W^{(k)} = W_{t+1}^{(k)} - W_t^{(k)}$ obtained by differencing consecutive checkpoints. We define the element-wise *nonzero raio* ρ across the entire model as

$$\rho = \frac{1}{\Sigma_{k=1}^{K}|W^{(k)}|} \sum_{k=1}^{K} ||\Delta W^{(k)}||_0$$

  In simple terms it is:

$$\rho = \frac{sum_of_changed_weights}{sum_of_all_weights}$$

This ratio is consistently low across different model families, around 1~2%.



(a) Visualization of weight update distribution

> Visualization of weight update distribution, demonstrating the sparse nature of parameter updates.

The update ratio remains stable, rapidly falling below 1% and staying there across 800 steps. With this, no need to send all 16GB of weights, but only 1~2%, which is only around 220MB of data.

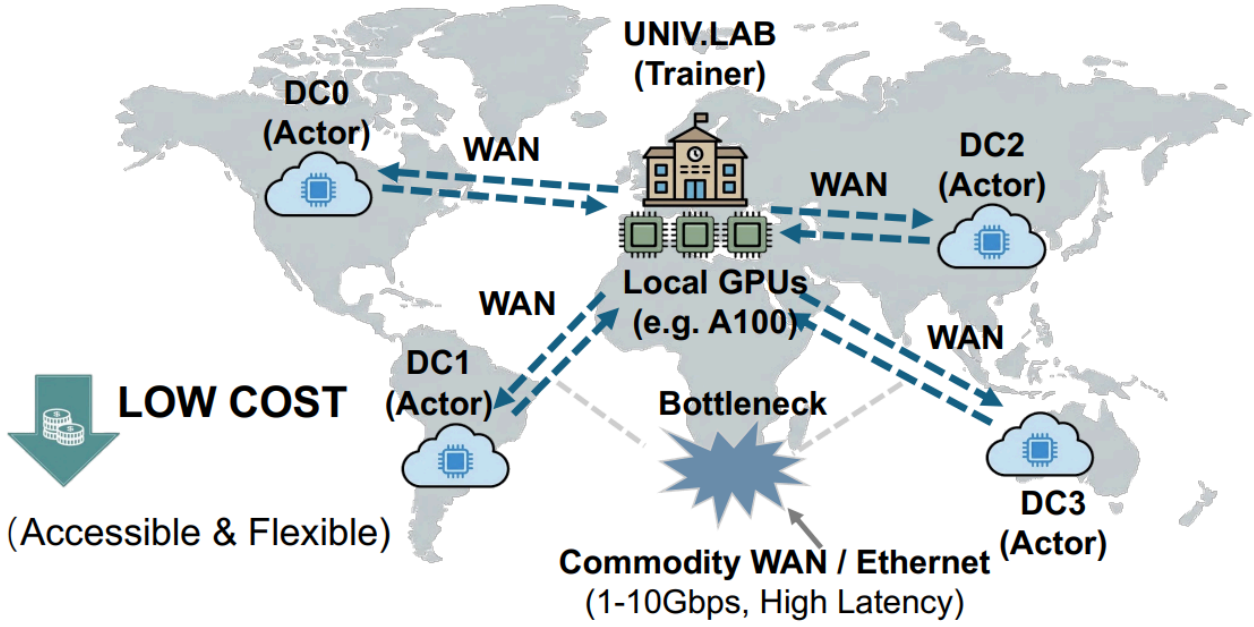
Overview of AuroraRL:

- **Deployment model:**

  The Trainer Hub connects to regional Relays over cross-cloud links up to 10 Gbps.

- **System components:**

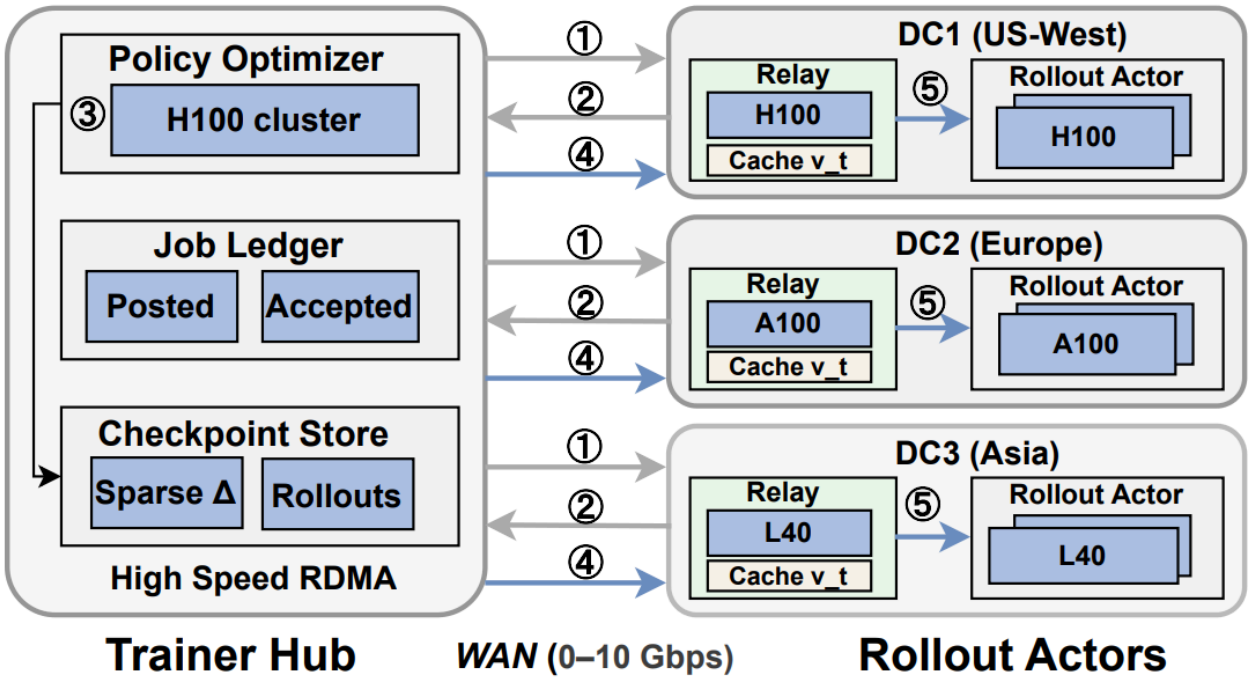
  - The Trainer Hub runs a policy optimizer (e.g. GRPO) on a cluster (e.g. RDMA-connected H100s), maintains a Job Ledger, and operates Checkpoint Store.
  - Rollout Actors span heterogeneous hardware across geographic regions (H100, A100, and L40 clusters).



(b) Loosely coupled GPU cluster over commodity link

- Relay caches the current policy version ν_t and forwards deltas to peer Actors in its region, reducing cross-region from $O(N)$ to one per region.

- **Workflow:**



Anchors:

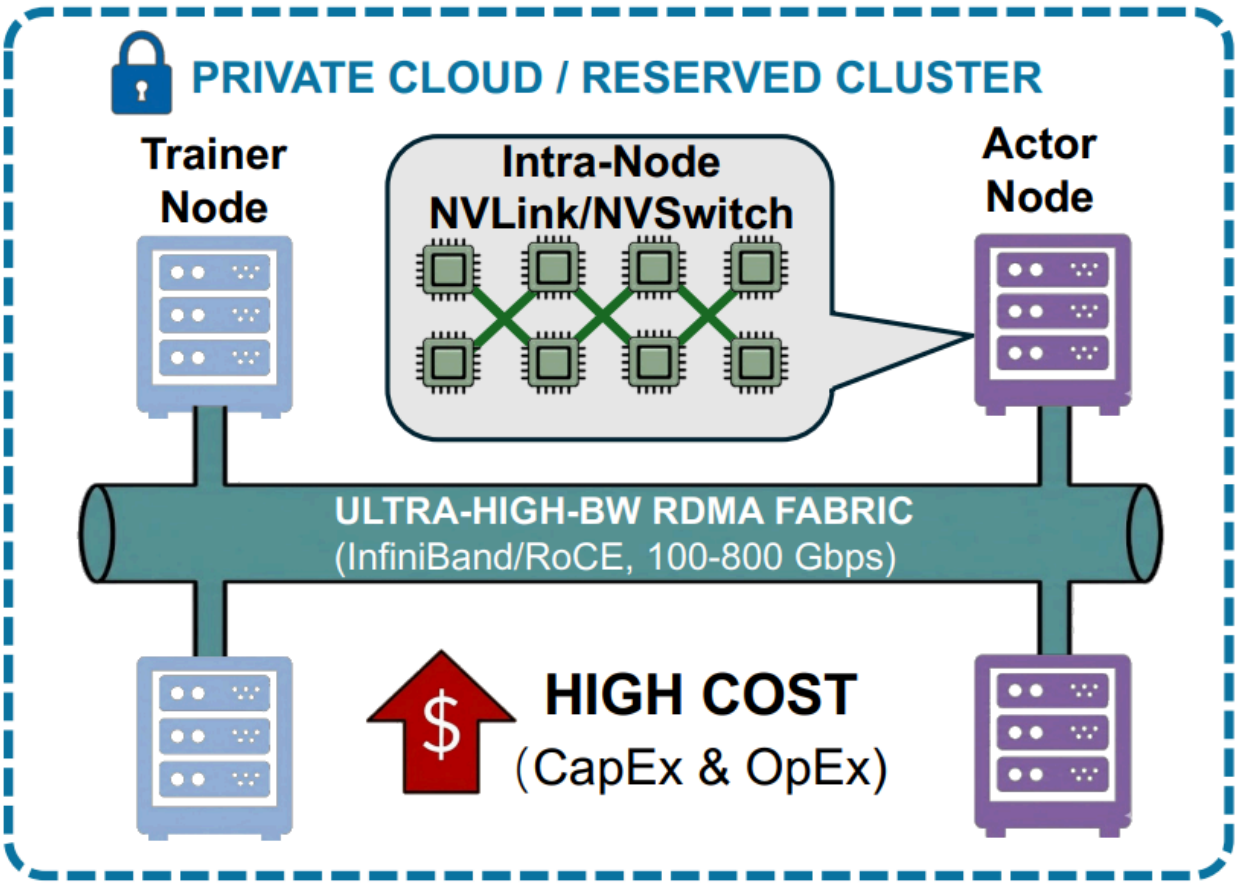
trainer-actor

- The Actor (The Generator):
  - Job: Runs inference to generate outputs based on the current model weights.

- The Trainer (The Learner):
  - Job: Collects the outputs and rewards generated by the actors, calculates the loss, and preforms backpropagation to update the model weights.

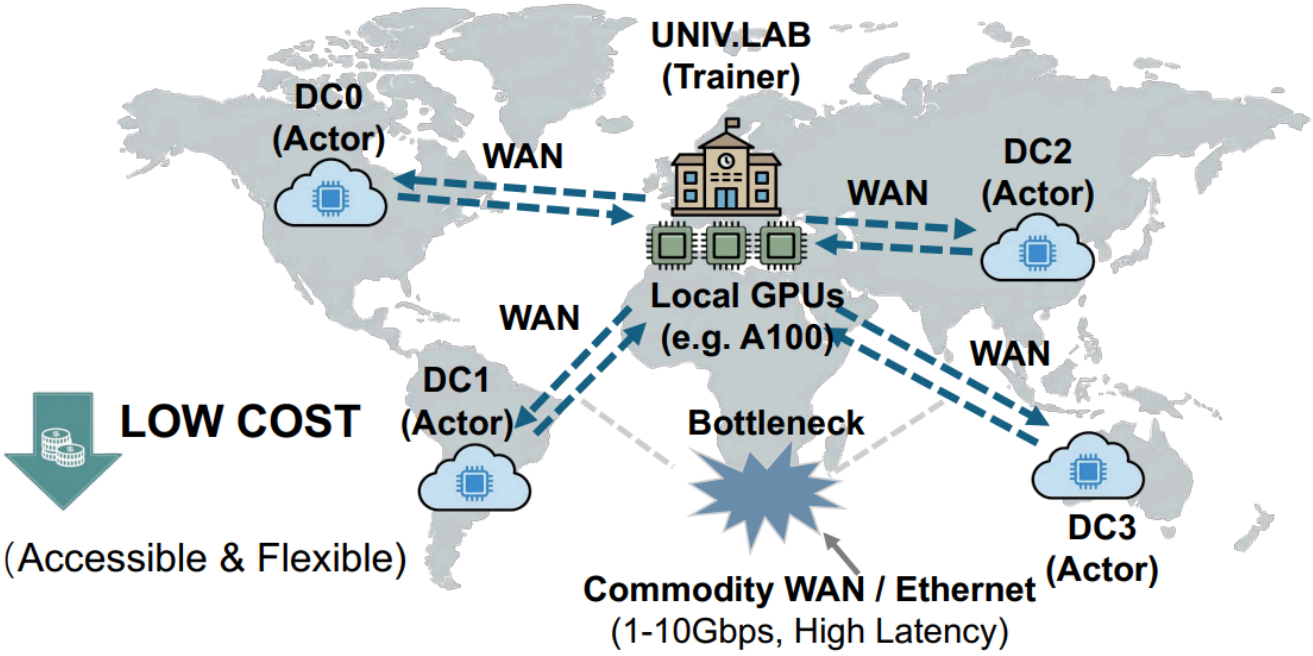
tightly-loosely-coupled:

- Tightly-Coupled cluster: System with RDMA-connected GPU HPC clusters.



(a) Tightly coupled GPU HPC cluster with RDMA

- Loosely-Coupled cluster: GPUs over standard Ethernet and WAN links.



(b) Loosely coupled GPU cluster over commodity link

rdma-hpc

In HPC (High-Performance Computing), RDMA (Remote Direct Memory Access) is a networking technology that allows computers in a cluster to r/w directly from each other's main memory without OS, CPU, or network stack.

bf16

The truncated version of float32: 1 sign bit, 8 the exponent bits, 7 mantissa bits. Unlike FP16, which has more precision, but less range, the BF16 has wider range, but less precision.

trainer-hub

The main server to update weights and distribute the deltas.

relay

A dual-role node that both generates rollouts and serves as regional proxy.

grpo

GRPO (Group Relative Policy Optimization) is a RL algorithm. Core innovation is that eliminates the memory-heave **Critic model**, making post-training for reasoning tasks much more memory-efficient. For any given prompt q, it runs the following steps:

- i. Group sampling: The model generates a group of G different responses for the same question.

- ii. Reward Calculation: Each response is evaluated and given a reward score r_i using rule-based checks - like validating a math answer or running code - or a reward model.
- iii. Group Relative Advantage: It calculates the mean and stadnard deviation of rewards within that specific group:

$$Advantage_i = \frac{r_i - mean(r)}{std(r)}$$

- iv. Policy Update: Responses that scored above the groups average get positive advantages (the model is more like to generate them in the future), while responses blow the group average get negative advantages get negative advantages.

job-ledger

Tracks posted and accepted work.

checkpoint-store

Holds versioned sparse deltas and collected rollouts.